Sessions and Memory: Context Engineering for Stateful AI

The white paper, **Context Engineering: Sessions, Memory** (November 2025), authored by **Kimberly Milam and Antonio Gulli**, explores how Sessions and Memory are essential for building stateful, intelligent LLM agents.

I. Context Engineering (CE)

Context Engineering is the practice of dynamically assembling and managing information within an LLM's context window to enable stateful, intelligent agents.

Core Principles:

- **Addressing Statelessness:** Large Language Models (LLMs) are inherently stateless outside of their training data, confining their reasoning to the context window of a single API call. CE constructs the necessary context for every turn of a conversation.
- **Evolution from Prompt Engineering:** Traditional Prompt Engineering focuses on crafting optimal, often static, system instructions. Conversely, Context Engineering addresses the **entire payload**, dynamically constructing a state-aware prompt based on the user, conversation history, and external data.
- **Goal:** The primary goal is to ensure the model has no more and no less than the most relevant information to complete its task.
- **Analogy:** CE is likened to the *mise en place* for an agent—gathering and preparing all necessary ingredients and tools before "cooking" (generating a response).

Components of the Context Payload:

Context Engineering governs the assembly of a complex payload

Component Category	Contents	Purpose
Context to Guide Reasoning	System Instructions, Tool Definitions, Few-Shot Examples	Defines the agent's persona, capabilities, constraints, and reasoning patterns.
Evidential & Factual Data	Long-Term Memory, External Knowledge (via RAG), Tool Outputs, Sub-Agent Outputs, Artifacts	Provides the substantive data the agent reasons over, serving as the 'evidence' for the response.
Immediate Conversational Information	Conversation History, State / Scratchpad, User's Prompt	Grounds the agent in the current interaction and defines the immediate task.

The Context Management Cycle:

Context Engineering manifests as a continuous cycle within the agent's operational loop for each conversational turn:

1. **Fetch Context:** The agent retrieves context, such as user memories, RAG documents, and recent conversation events, often dynamically using the user query and metadata.
2. **Prepare Context:** The agent framework dynamically constructs the full prompt for the LLM call. This is a **blocking, "hot-path" process**.
3. **Invoke LLM and Tools:** The agent calls the LLM and necessary tools, appending tool and model output to the context.
4. **Upload Context:** New information gathered during the turn is uploaded to persistent storage, often as a **"background" process** (asynchronously) for memory consolidation or post-processing.

II. Sessions

A session is a foundational element of Context Engineering that encapsulates the immediate dialogue history and working memory for a single, continuous conversation.

Structure and Function:

- **Definition:** The container for an entire conversation with an agent, holding the chronological history of the dialogue and the agent's working memory.
- **Components:** Every session contains the **chronological history (events)** and the agent's **working memory (state)**.
    - **Events:** The turn-by-turn building blocks, including user input, agent response, tool calls, and tool output. The structure is analogous to the list of Content objects passed to the Gemini API.
    - **State / Scratchpad:** Temporary, structured data relevant to the current conversation (e.g., items in a shopping cart).
- **Storage:** Since production agents are typically stateless, the conversation history must be saved to persistent storage (e.g., robust databases or managed solutions like Agent Engine Sessions) to maintain a continuous user experience.

Managing Long Context Conversation:

As context grows, costs and latency increase, and models can suffer from "context rot". Compaction strategies intelligently trim history while preserving vital information.

Strategy	Description
Keep the last N turns	The simplest strategy, using a "sliding window" to discard everything older than the most recent N turns.
Token-Based Truncation	Counting tokens backward from the most recent message and cutting off the conversation when a predefined token limit (e.g., 4000 tokens) is met.
Recursive Summarization	Older parts of the conversation are replaced by an LLM-generated summary. This summary is often prefixed to the more recent, verbatim messages. This expensive operation should be performed asynchronously in the background and persisted.

Compaction can be triggered by Count-Based Triggers (token size or turn count), Time-Based Triggers (lack of activity), or Event-Based Triggers (semantic or task completion).

Multi-Agent Systems and Interoperability:

In multi-agent systems, agents must share session history, which can be handled in two main ways:

1. **Shared, Unified History:** All agents read and write all events to a single, central conversation log. This is best for tightly coupled, collaborative tasks requiring a single source of truth.
2. **Separate, Individual Histories:** Each agent maintains its own private log, functioning as a black box. Communication occurs only through explicit messages (e.g., Agent-as-a-Tool or Agent-to-Agent Protocol) sharing final outputs, not internal processes.

A critical trade-off is that the abstraction provided by agent frameworks (e.g., ADK vs. LangGraph) isolates agents using different frameworks, making session records non-portable due to framework-specific schemas. **Memory** offers a more robust solution for interoperability by abstracting shared knowledge into a framework-agnostic data layer designed to hold processed, canonical information.

Production Considerations for Sessions:

- **Security and Privacy:** Requires **strict isolation** (via ACLs) to ensure one user cannot access another's session data. **PII redaction** (e.g., using Model Armor) is a best practice before data is written to storage.

- **Data Integrity:** Maintaining a deterministic, correct chronological sequence of events is fundamental. Inactive sessions should be managed with a Time-to-Live (TTL) policy.
- **Performance:** Reading and writing session history must be extremely fast, as it is on the "hot path" of every user interaction. Latency is mitigated by filtering or compacting history before transfer.

III. Memory

Memory is the mechanism for long-term persistence, capturing and consolidating key information across multiple sessions to provide a continuous and personalized experience for LLM agents. It is a snapshot of extracted, meaningful information.

Memory vs. RAG:

Memory managers and RAG engines fulfill distinct and complementary roles:

- **RAG Engines:** Focus on injecting external, factual knowledge (static, shared, authoritative data). **RAG makes an agent an expert on facts**. (Analogy: The research librarian).
- **Memory Managers:** Focus on creating a personalized and stateful experience (dynamic, highly isolated, user-specific data derived from dialogue). **Memory makes the agent an expert on the user**. (Analogy: The personal assistant).

Memory Generation Process:

Memory generation autonomously transforms raw conversational data into structured, meaningful insights using an LLM-driven ETL (Extract, Transform, Load) pipeline.

The process generally follows four stages:

1. **Ingestion:** Client provides raw data (typically session history) to the memory manager.
2. **Extraction & Filtering:** An LLM extracts meaningful content, capturing only information that fits predefined topic definitions (using schemas or natural language definitions).
3. **Consolidation (Self-Curation):** The most sophisticated stage. The LLM compares newly extracted information with existing memories to handle conflict resolution, duplication, and evolution of facts. This leads to operations like MERGE, DELETE/INVALIDATE, or CREATE.
4. **Storage:** The updated memory is persisted to a durable storage layer (vector database or knowledge graph).

Types and Structure:

- **Declarative Memory ("Knowing What"):** Knowledge of facts, figures, and events, including Semantic (world knowledge) and Entity/Episodic (specific user facts).
- **Procedural Memory ("Knowing How"):** Knowledge of skills and workflows, guiding the agent's actions (e.g., correct sequence of tool calls). Procedural memory management requires specialized algorithms distinct from declarative memory retrieval.
- **Content and Metadata:** A single memory typically consists of **Content** (structured or unstructured, framework-agnostic data) and **Metadata** (context like ID, owner, source labels).
- **Multimodal Memory:** Typically, the source data is multimodal (text, image, audio), but the memory content itself is a textual insight derived from the source, as generating and retrieving unstructured binary data is complex.

Organization and Storage:

- **Organization Patterns:**
  - **Collections:** Multiple self-contained, natural language memories for a single user.
  - **Structured User Profile:** A set of core facts about a user, continuously updated for quick lookups.
  - **Rolling Summary:** Consolidates all information into a single, evolving natural-language summary.
- **Storage Architectures:**
  - **Vector Databases:** Most common approach; enables retrieval based on semantic similarity using embedding vectors.
  - **Knowledge Graphs:** Stores memories as a network of entities and relationships, ideal for structured, relational queries.
  - **Hybrid Approach:** Combines vector embeddings with knowledge graphs for both relational and semantic searches.

Provenance and Trustworthiness:

Provenance is the detailed record of a memory's origin and history, critical for evaluating its quality and resolving conflicts.

- **Source Types:** Influence trust level (e.g., Bootstrapped Data from a CRM is high-trust; Implicitly extracted User Input is generally less trustworthy; Tool Output is often brittle/stale).

- **Conflict Resolution:** Memory managers use a hierarchy of trust (prioritizing trusted sources or most recent information) when sources contradict each other.
- **Dynamic Confidence:** Confidence in a memory increases through corroboration (consistent information from multiple sources) and decreases (decays) over time. The confidence score is injected into the prompt for the LLM to weigh evidence during inference.
- **Memory Pruning (Forgetting):** An active curation process to discard memories that are no longer useful, triggered by time-based decay, low confidence, or irrelevance.

Retrieval and Inference:

- **Retrieval Strategy:** Should blend scores based on **Relevance** (semantic similarity), **Recency** (time-based), and **Importance** (significance) to find the best fit.
- **Timing for Retrieval:**
    - **Proactive Retrieval:** Memories are automatically loaded at the start of every turn. Ensures context is always available but adds latency.
    - **Reactive Retrieval (Memory-as-a-Tool):** The agent uses a tool to query its memory, deciding when context is necessary. More efficient but requires an additional LLM call.
- **Placement for Inference:**
    - **System Instructions:** Memories are appended alongside a preamble, giving them high authority. Ideal for stable, global information like user profiles.
    - **Conversation History:** Memories are injected directly into the dialogue (e.g., before the history or via tool outputs). This risks dialogue injection, where the model treats the memory as an actual utterance.

Production and Security:

- **Asynchronous Generation:** Memory generation must be handled asynchronously as a **background process** (non-blocking API calls) to avoid blocking the user experience, as it is computationally expensive.
- **Scalability and Resilience:** Requires robust message queues to buffer high volumes of events, retry mechanisms for transient failures, and database multi-region replication for low latency and high availability.
- **Privacy and Security:** Memory requires stringent controls, including strict isolation per user (via ACLs), user control over their data (opt-out), PII redaction, and sanitization (using tools like Model Armor) to prevent **memory poisoning** (corrupting the agent's knowledge base via prompt injection).

To visualize the relationship between the two main concepts, think of the agent's Session as the **desk** where all the immediate work, notes, and documents for the current project are spread out (temporary, messy, high-latency concern), while Memory is the meticulously organized **filing cabinet** where only the most critical, processed, and finalized documents are stored for long-term use across all future projects (persistent, curated, low-latency retrieval concern).